Exp 12

Part A: Echo Client-Server using TCP Sockets

AIM

To write and implement an Echo Client-Server program using TCP sockets in C.

Algorithm

Server Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Put the server socket in listening mode using listen().
4. Accept an incoming client connection using accept().
5. Read the message sent by the client using read() or recv().
6. Echo the same message back to the client using write() or send().
7. Close the client and server sockets using close().

Client Side

1. Create a TCP socket using socket(AF_INET, SOCK_STREAM, 0).
2. Connect to the server's IP address and port number using connect().
3. Prompt the user for input and send the message to the server using write() or send().

4. Receive the echoed message back from the server using read() or recv().
5. Display the received message on the terminal.
6. Close the socket using close().

Code Implementation (TCP)

`tcp_server.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // 1. Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
```

```
        exit(EXIT_FAILURE);
    }

    // 2. Bind socket
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    // 3. Listen for connections
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }

    printf("[TCP Server] Waiting for incoming connections on port
%d...\n", PORT);

    // 4. Accept connection
```

```
    if ((new_socket = accept(server_fd, (struct sockaddr
*)&address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        exit(EXIT_FAILURE);
    }

    // 5 & 6. Read and Echo back
    read(new_socket, buffer, BUFFER_SIZE);
    printf("[TCP Server] Received from client: %s\n", buffer);

    send(new_socket, buffer, strlen(buffer), 0);
    printf("[TCP Server] Echoed message back to client.\n");

    // 7. Close sockets
    close(new_socket);
    close(server_fd);
    return 0;
}
```

tcp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
```

```
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];

    // 1. Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 address from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <=
0) {
        printf("\nInvalid address / Address not supported \n");
        return -1;
```

```
    }

    // 2. Connect to server
    if (connect(sock, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }

    // 3. Send message
    printf("Enter message to send to server: ");
    fgets(message, BUFFER_SIZE, stdin);
    message[strcspn(message, "\n")] = 0; // Remove newline

    send(sock, message, strlen(message), 0);
    printf("[TCP Client] Message sent.\n");

    // 4 & 5. Receive echoed message and display
    read(sock, buffer, BUFFER_SIZE);
    printf("[TCP Client] Echo received from server: %s\n", buffer);

    // 6. Close socket
    close(sock);
    return 0;
```

}

Input/Output (TCP)

Input (Client Terminal):

Enter message to send to server: Hello Networks Lab!

Output (Server Terminal):

[TCP Server] Waiting for incoming connections on port 8080...

[TCP Server] Received from client: Hello Networks Lab!

[TCP Server] Echoed message back to client.

Output (Client Terminal):

[TCP Client] Message sent.

[TCP Client] Echo received from server: Hello Networks Lab!

Part B: Echo Client-Server using UDP Sockets

AIM

To write and implement an Echo Client-Server program using UDP sockets in C.

Algorithm

Server Side

1. Create a UDP socket using socket(AF_INET, SOCK_DGRAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Receive a message and store the client's address using recvfrom().
4. Echo the message back to the client using sendto().

5. Close the socket using close().

Client Side

1. Create a UDP socket using socket(AF_INET, SOCK_DGRAM, 0).
2. Get input from the user and send the message to the server using sendto().
3. Receive the echoed message back from the server using recvfrom().
4. Display the received message on the terminal.
5. Close the socket using close().

Code Implementation (UDP)

udp_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE] = {0};
    struct sockaddr_in servaddr, cliaddr;
```

```
    socklen_t len = sizeof(cliaddr);

    // 1. Create socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    // 2. Bind socket
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    if (bind(sockfd, (const struct sockaddr *)&servaddr,
sizeof(servaddr)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    printf("[UDP Server] Waiting for data on port %d...\n", PORT);
```

```
    // 3. Receive message
    int n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE,
MSG_WAITALL, (struct sockaddr *)&cliaddr, &len);
    buffer[n] = '\0';
    printf("[UDP Server] Received from client: %s\n", buffer);

    // 4. Echo back message
    sendto(sockfd, (const char *)buffer, strlen(buffer),
MSG_CONFIRM, (const struct sockaddr *)&cliaddr, len);
    printf("[UDP Server] Echoed message back to client.\n");

    // 5. Close socket
    close(sockfd);
    return 0;
}
```

udp_client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024
```

```
int main() {
    int sockfd;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];
    struct sockaddr_in servaddr;
    socklen_t len = sizeof(servaddr);

    // 1. Create socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = INADDR_ANY;

    // 2. Get input and send
    printf("Enter message to send to UDP server: ");
    fgets(message, BUFFER_SIZE, stdin);
    message[strcspn(message, "\n")] = 0;
```

```
    sendto(sockfd, (const char *)message, strlen(message),
MSG_CONFIRM, (const struct sockaddr *)&servaddr,
sizeof(servaddr));
    printf("[UDP Client] Message sent.\n");

    // 3 & 4. Receive echo and display
    int n = recvfrom(sockfd, (char *)buffer, BUFFER_SIZE,
MSG_WAITALL, (struct sockaddr *)&servaddr, &len);
    buffer[n] = '\0';
    printf("[UDP Client] Echo received from server: %s\n",
buffer);

    // 5. Close socket
    close(sockfd);
    return 0;
}
```

Input/Output (UDP)

Input (Client Terminal):

Enter message to send to UDP server: Testing UDP Socket

Output (Server Terminal):

[UDP Server] Waiting for data on port 8080...

[UDP Server] Received from client: Testing UDP Socket

[UDP Server] Echoed message back to client.

Output (Client Terminal):

[UDP Client] Message sent.

[UDP Client] Echo received from server: Testing UDP Socket